

Python Dictionary Objects

Dictionaries are simple mappings that associate some hashable objects to some objects.

To create a dictionary in Python, write some `key:value` pairs between braces seperating them with `,`. Here

- `key` must be a hashable Python object like: string, integer, floating point number, or tuple and
- `value` is any Python object.

In [19]:
```
d = {
    'name':'Ali',
    'surname':'Cin',
    'age':17,
    'address':{
     'country':'Türkiye',
     'address': 'Antalya',
     },
    'friends':['Ahmet','Mehmet'],
    17: 'ok'
    }
```

Dictionary keys must be unique and hashable. But values can be any Python object so they can be nested.

You can create a dictionary with an iterable of `key,value` pairs using `dict()` function.

In [20]:
```
dict(((3,9),(7,49),(1,1)))
```

Out[20]:
```
{3: 9, 7: 49, 1: 1}
```

Or you can create a dictionary with comprehension syntax.

In [21]:
```
{x:x**2 for x in range(10) if x & 1}
```

Out[21]:
```
{1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
```

In comprehension `if` part is optional.

In [22]:
```
{x: x**2 for x in range(10)}
```

Out[22]:
```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8: 64, 9: 81}
```

Empty Dictionary

Empty dictionary can be created by `{}` or `dict()`.

Accessing The Items of Dictionary

In [23]:
```
d['address']['country']
```

Out[23]:
```
'Türkiye'
```

In [24]:
```
d[17]
```

Out[24]:
```
'ok'
```

To test if a `key` exists in a dictionary use `in` keyword.

In [25]:
```
'job' in d
```

Out[25]:
```
False
```

If the `key` does not exist Python will yield a `KeyError`. To prevent this, you can use `.get(key)` method. `.get(key)` method gives `None` if the key is not in the dictionary.

In [26]:
```
d.get('job')
```

Modifying The Dictionary

You can change the values and also add new `key:value` pairs to a dictionary.

In [27]:
```
d['age'] = 18
d['gender'] = 'male'
d
```

Out[27]:
```
{'name': 'Ali',
 'surname': 'Cin',
 'age': 18,
 'address': {'country': 'Türkiye', 'address': 'Antalya'},
 'friends': ['Ahmet', 'Mehmet'],
 17: 'ok',
 'gender': 'male'}
```

In [28]:
```
d.update({'age':19})
d
```

Out[28]:
```
{'name': 'Ali',
 'surname': 'Cin',
 'age': 19,
 'address': {'country': 'Türkiye', 'address': 'Antalya'},
 'friends': ['Ahmet', 'Mehmet'],
 17: 'ok',
 'gender': 'male'}
```

Use `.setdefault(key,value)` to add a `key:value` pair in to the dictionary if the `key` does not exists. If it is already in the dictionary this method gets that value.

In [29]:
```
d.setdefault('friends',[])
```

Out[29]:
```
['Ahmet', 'Mehmet']
```

To remove a `key:value` pair use `del` keyword.

In [30]:
```
del d[17]
d
```

Out[30]:
```
{'name': 'Ali',
 'surname': 'Cin',
 'age': 19,
 'address': {'country': 'Türkiye', 'address': 'Antalya'},
 'friends': ['Ahmet', 'Mehmet'],
 'gender': 'male'}
```

The `.clear()` method remove all items from the dictioanry. And the `.copy()` method makes a copy of the dictionary.

Geting The List of Keys, Values And Key-Value Pairs

In [31]:
```
d.keys()
```

Out[31]:
```
dict_keys(['name', 'surname', 'age', 'address', 'friends', 'gender'])
```

In [32]:
```
d.values()
```

Out[32]:
```
dict_values(['Ali', 'Cin', 19, {'country': 'Türkiye', 'address': 'Antalya'}, ['Ahme
t', 'Mehmet'], 'male'])
```

In [33]:
```
d.items()
```

Out[33]:
```
dict_items([('name', 'Ali'), ('surname', 'Cin'), ('age', 19), ('address', {'country':
'Türkiye', 'address': 'Antalya'}), ('friends', ['Ahmet', 'Mehmet']), ('gender', 'mal
e')])
```